Cloudflare Tunnel Reverse Proxy - Complete Setup Guide

Table of Contents

Prerequisites

- A Cloudflare account (free tier works)
- A domain managed by Cloudflare (for named tunnels with custom domains)
- Linux server with the service you want to expose

Installation

Option 1: Download Binary (Recommended)

```
# Download the latest cloudflared binary
wget https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64

# Make it executable
chmod +x cloudflared-linux-amd64

# Move to system path
sudo mv cloudflared-linux-amd64 /usr/local/bin/cloudflared

# Verify installation
cloudflared --version
```

Option 2: Package Manager (Debian/Ubuntu)

```
# Add Cloudflare's GPG key
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg | sudo tee /usr/share/keyrings/cloudflare-main.gpg >/dev/null

# Add the repository
echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare.com/cloudflared focal main' | sudo tee /etc/apt/sources.list.d/cloudflared.list

# Install
sudo apt update && sudo apt install cloudflared
```

Quick Tunnel (Temporary)

Quick tunnels are perfect for testing or temporary access. They create a random `*.trycloudflare.com` subdomain.

Basic Usage

```
# Expose a local HTTP service on port 8080
cloudflared tunnel --url http://localhost:8080

# Expose HTTPS
cloudflared tunnel --url https://localhost:8443

# Expose with WebSocket support (auto-detected, but can be explicit)
cloudflared tunnel --url http://localhost:8080
```

Example Output

```
2025-12-22T10:00:00Z INF Starting tunnel
2025-12-22T10:00:01Z INF +--------------------------------------------------------------+
2025-12-22T10:00:01Z INF |  Your quick tunnel URL is: https://random-words.trycloudflare.com  |
2025-12-22T10:00:01Z INF +--------------------------------------------------------------+
```

> ***Note**: Quick tunnels are temporary. The URL changes each time you restart.*

Named Tunnel (Persistent)

Named tunnels provide a permanent tunnel ID and can be mapped to your own domain.

Step 1: Authenticate

```
# Login to Cloudflare (opens browser)
cloudflared tunnel login

# This creates ~/.cloudflared/cert.pem
```

Step 2: Create a Named Tunnel

```
# Create tunnel (replace 'my-tunnel' with your preferred name)
cloudflared tunnel create my-tunnel

# This creates:
# - Tunnel ID (UUID)
# - Credentials file: ~/.cloudflared/<TUNNEL_ID>.json
```

Step 3: Configure the Tunnel

Create a config file at `~/.cloudflared/config.yml`:

```
# ~/.cloudflared/config.yml

tunnel: my-tunnel  # or use the UUID
credentials-file: /home/YOUR_USER/.cloudflared/<TUNNEL_ID>.json

ingress:
  # Route traffic for your domain to local service
  - hostname: app.yourdomain.com
    service: http://localhost:8080

  # WebSocket service example
  - hostname: ws.yourdomain.com
    service: http://localhost:8081
    originRequest:
      noTLSVerify: true

  # HTTPS backend
  - hostname: secure.yourdomain.com
    service: https://localhost:8443
    originRequest:
      noTLSVerify: true  # Skip TLS verification for self-signed certs

  # Catch-all (required - returns 404 for unmatched)
  - service: http_status:404
```

Step 4: Create DNS Record

```
# Route your subdomain through the tunnel
cloudflared tunnel route dns my-tunnel app.yourdomain.com

# This creates a CNAME record pointing to <TUNNEL_ID>.cfargotunnel.com
```

Step 5: Run the Tunnel

```
# Run with config file
cloudflared tunnel run my-tunnel

# Or run directly without config
cloudflared tunnel --config ~/.cloudflared/config.yml run
```

Systemd Service (Autostart)

Generic Template

Create `/etc/systemd/system/cloudflared-tunnel.service`:

```
[Unit]
Description=Cloudflare Tunnel
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=YOUR_USERNAME
Group=YOUR_USERNAME

# For Quick Tunnel (random URL)
# ExecStart=/usr/local/bin/cloudflared tunnel --url http://localhost:8080

# For Named Tunnel (recommended)
ExecStart=/usr/local/bin/cloudflared tunnel --config /home/YOUR_USERNAME/.cloudflared/config.yml run

Restart=always
RestartSec=5

# Logging
StandardOutput=journal
StandardError=journal
SyslogIdentifier=cloudflared

# Security hardening
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=/home/YOUR_USERNAME/.cloudflared

[Install]
WantedBy=multi-user.target
```

Complete Service with Local App (Example)

This example runs both your app and the tunnel together:

Create `/etc/systemd/system/app-with-tunnel.service`:

```
[Unit]
Description=My App with Cloudflare Tunnel
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=escort
Group=escort
WorkingDirectory=/home/escort/WhisperLiveKit

# Script that starts both app and tunnel
ExecStart=/home/escort/WhisperLiveKit/start_with_tunnel.sh
ExecStop=/bin/kill -TERM $MAINPID

Restart=always
RestartSec=10

StandardOutput=journal
StandardError=journal
SyslogIdentifier=app-tunnel

[Install]
WantedBy=multi-user.target
```

Startup Script (`start_with_tunnel.sh`)

```
#!/bin/bash

# Configuration
APP_PORT=8080
TUNNEL_NAME="my-tunnel"
LOG_DIR="/home/escort/logs"

mkdir -p "$LOG_DIR"

# Cleanup function
cleanup() {
    echo "Shutting down..."
    kill $APP_PID 2>/dev/null
    kill $TUNNEL_PID 2>/dev/null
    exit 0
}

trap cleanup SIGTERM SIGINT

# Start your application
echo "Starting application on port $APP_PORT..."
cd /home/escort/WhisperLiveKit
source venv/bin/activate
python -m whisperlivekit.basic_server --port $APP_PORT >> "$LOG_DIR/app.log" 2>&1 &
APP_PID=$!

# Wait for app to be ready
sleep 5

# Start Cloudflare tunnel
echo "Starting Cloudflare tunnel..."
cloudflared tunnel --config /home/escort/.cloudflared/config.yml run >> "$LOG_DIR/tunnel.log" 2>&1 &
TUNNEL_PID=$!

echo "App PID: $APP_PID, Tunnel PID: $TUNNEL_PID"
```

```
# Wait for either process to exit
wait -n
cleanup
```

Enable and Start Service

```
# Reload systemd
sudo systemctl daemon-reload

# Enable autostart on boot
sudo systemctl enable cloudflared-tunnel.service

# Start the service
sudo systemctl start cloudflared-tunnel.service

# Check status
sudo systemctl status cloudflared-tunnel.service

# View logs
sudo journalctl -u cloudflared-tunnel.service -f
```

Common Use Cases

Expose Web Server (HTTP)

```
# Quick
cloudflared tunnel --url http://localhost:80

# Config file
ingress:
  - hostname: web.example.com
    service: http://localhost:80
  - service: http_status:404
```

Expose API with WebSocket Support

```
# Quick
cloudflared tunnel --url http://localhost:8081

# Config file (WebSockets work automatically)
ingress:
  - hostname: api.example.com
    service: http://localhost:8081
  - service: http_status:404
```

Expose Multiple Services

```
# ~/.cloudflared/config.yml
tunnel: multi-service-tunnel
credentials-file: /home/user/.cloudflared/<UUID>.json

ingress:
  - hostname: app.example.com
    service: http://localhost:3000

  - hostname: api.example.com
    service: http://localhost:8080

  - hostname: grafana.example.com
    service: http://localhost:3001

  - service: http_status:404
```

Expose SSH (TCP)

```
ingress:
  - hostname: ssh.example.com
    service: ssh://localhost:22
  - service: http_status:404
```

Client connects via:

```
cloudflared access ssh --hostname ssh.example.com
```

Expose with Access Control (Zero Trust)

```
ingress:
  - hostname: private.example.com
    service: http://localhost:8080
    originRequest:
      access:
        required: true
        teamName: your-team
  - service: http_status:404
```

Management Commands

```
# List all tunnels
cloudflared tunnel list

# Get tunnel info
cloudflared tunnel info my-tunnel

# Delete a tunnel
cloudflared tunnel delete my-tunnel

# Clean up unused connections
cloudflared tunnel cleanup my-tunnel

# Route DNS
cloudflared tunnel route dns my-tunnel subdomain.example.com

# Delete DNS route
cloudflared tunnel route dns -d my-tunnel subdomain.example.com

# Check tunnel health
curl -s https://your-tunnel-url/cdn-cgi/trace
```

Troubleshooting

Common Issues

1. "failed to sufficiently increase receive buffer size"

```
# Fix by increasing UDP buffer (temporary)
sudo sysctl -w net.core.rmem_max=2500000

# Permanent fix - add to /etc/sysctl.conf
echo "net.core.rmem_max=2500000" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

2. Tunnel connects but site unreachable

- Check if your local service is running
- Verify the port number in config
- Check firewall rules: `sudo ufw status`

3. "certificate has expired"

```
# Re-authenticate
cloudflared tunnel login
```

4. Connection drops frequently

- Check network stability
- Increase `retries` in config:

```
tunnel: my-tunnel
credentials-file: /path/to/creds.json
retries: 10
grace-period: 30s
```

5. WebSocket not working

- Ensure your app handles WebSocket upgrade correctly
- Check if proxy headers are being passed:

```
ingress:
  - hostname: ws.example.com
    service: http://localhost:8080
    originRequest:
      httpHostHeader: ws.example.com
```

Useful Debug Commands

```
# Run with debug logging
cloudflared tunnel --loglevel debug run my-tunnel

# Test connectivity
cloudflared tunnel --url http://localhost:8080 --loglevel debug

# Check tunnel metrics
curl http://localhost:60123/metrics  # If metrics endpoint enabled
```

Quick Reference

Command	Description
`cloudflared tunnel login`	Authenticate with Cloudflare
`cloudflared tunnel create NAME`	Create new named tunnel
`cloudflared tunnel list`	List all tunnels
`cloudflared tunnel run NAME`	Start a tunnel
`cloudflared tunnel route dns NAME HOST`	Create DNS route
`cloudflared tunnel delete NAME`	Delete a tunnel
`cloudflared tunnel --url http://localhost:PORT`	Quick tunnel

Security Best Practices

1. **Use named tunnels** for production (credentials stay local)
2. **Enable Cloudflare Access** for sensitive services
3. **Restrict service binding** - bind to `127.0.0.1` not `0.0.0.0`
4. **Use HTTPS** between cloudflared and your app when possible
5. **Rotate credentials** periodically
6. **Monitor tunnel logs** for suspicious activity

Additional Resources

- Cloudflare Tunnel Documentation (https://developers.cloudflare.com/cloudflare-one/connections/connect-apps/)
- Zero Trust Setup Guide (https://developers.cloudflare.com/cloudflare-one/)
- Cloudflared GitHub (https://github.com/cloudflare/cloudflared)